Campus Resource & Activity Management System

Desktop GUI — Test Report

CS3400 Data Structures & Algorithms · Optional desktop edition

Group member	Student number
M.E.C. Pama	201404452
L.V. Soai	202322577
R. Mphatle	201902287

1. Test environment

Item	Detail
Operating system	Windows (x64)
GUI framework	Qt 6.8.1 Widgets
Compiler	MinGW 13.1.0 (g++), C++17
Packaging	windeployqt (Qt runtime + plugins bundled)
Build command	`qmake campus_gui.pro && mingw32-make -j4`
Core logic under test	The project's own data-structure / algorithm source, linked unmodified

2. Build verification

- The GUI compiled and linked against the project's real `Resource / Request / Tree / HashTable / Graph / Searching / Utils` translation units and header-only templates — **0 errors**.
- `windeployqt` deployed `Qt6Core/Gui/Widgets/Network.dll`, `platforms\qwindows.dll`, image-format, TLS and style plugins, plus the MinGW runtime (`libgcc_s_seh-1 / libstdc++-6 / libwinpthread-1.dll`) into the release folder.
- The resulting folder `CS3400_Campus_Resource_Manager` is self-contained: the executable, all required DLLs/plugins and the `data\` files.
- No graded file was modified; all new code is confined to `gui\`.

3. Functional tests

Each screen was exercised on the running application with the supplied sample data (15 resources, 6 requests, 10 campus locations). Where the GUI reuses the console code, outputs were checked against the console app and matched.

#	Feature / structure	Action	Expected	Observed	Result
1	Launch (portable)	Double-click the packaged exe	App opens, loads data	Opened; status: Loaded 15 resources, 3 normal, 3 priority	PASS
2	Dashboard	Open Dashboard	Live counts from all structures	15 total, 12 available, 1 borrowed, 2 maintenance; 7/4/4 by type; 3 FIFO, 3 heap; 10 locations, 11 pathways	PASS
3	Resources (linked list)	List resources	Full table	B001–R202 with status, capacity, detail	PASS

4	AVL tree	Show four traversals	In-order sorted; height O(log n)	Nodes 15, Height 5; in-order B001…R202 — identical to console	PASS
5	Hash table	Show statistics	djb2 + chaining stats	53 buckets, load 0.28, 2 collisions, longest chain 2 — identical to console	PASS
6	FIFO queue	Open Requests	Oldest first	Request 1001 (Thabo Mokoena) at the front of the queue	PASS
7	Priority heap	Open Requests	Most urgent first	1005 (Head Librarian, Emergency) at the top of the heap	PASS
8	Graph — Dijkstra	Shortest path Main Library → Sports Centre	Least-distance route	450 m via Refectory — identical to console	PASS
9	Sorting / searching	DS & Algorithms benchmark	Measured timings, expected growth	Ran at 1k/5k/10k (see §4)	PASS

4. Performance benchmark (measured on the test machine)

Times are in microseconds of CPU time, for the three sorts and 1,000 lookups per method.

n	Insertion	Merge	Quick	Linear ×1000	Binary ×1000	AVL ×1000	Hash ×1000
1,000	11,000	1,000	0	22,000	1,000	1,000	0
5,000	243,000	6,000	4,000	69,000	2,000	0	1,000
10,000	1,025,000	8,000	15,000	247,000	0	5,000	0

Interpretation. Insertion sort grows roughly with n^2 (≈11 ms → ≈1,025 ms, ~93× for 10× the data), while merge and quick sort grow with n log n and stay in single-digit milliseconds. Linear search grows with n (≈22 ms → ≈247 ms, ~11×), whereas binary search, AVL and hash lookups stay almost flat.

> **Measurement note.** Timings use `clock()`, whose resolution on Windows is coarse (~1 ms), so the smallest values round to 0 or 1,000 µs. The growth *trends* are the meaningful result and match the theoretical complexity classes; absolute values vary between machines and runs.

5. Portability check

- The application was launched from the **packaged release folder** (not the build tree) and ran correctly with the bundled Qt and MinGW runtime — confirming the required DLLs and plugins are present.
- **Not yet verified:** launching on a *separate clean Windows PC with no Qt or compiler installed.* Testing was performed on the development machine only. The bundled runtime is expected to make it self-contained, but a clean-machine run has not been carried out.

6. Conclusion

The desktop GUI builds cleanly against the project's real data structures and algorithms, launches from a self-contained portable folder, and all seven screens operate correctly on live data — with the AVL traversals, hash statistics and Dijkstra result matching the console application exactly. The original console application and its files were not modified.